



Facultad De Ingeniería De Sistemas

Docente:

Luis Murillo

Integrantes:

Franklin Miranda

Sebastian Barragan

Luisa Chacón

José Ramos

Juan Rodriguez

03/09/2026

Secuencia de LEDs Controlada por Pulsador y Potenciómetro: Desarrollo por Fases

Control de dirección y velocidad de una secuencia de LEDs tipo semáforo, desarrollado y probado en tres fases independientes antes de su integración final, con Arduino IDE y temporización no bloqueante mediante millis().

1. Introducción

Este proyecto consiste en el control de una secuencia de LEDs (rojo, amarillo y verde) tipo semáforo, cuya velocidad y dirección pueden ser modificadas por el usuario en tiempo real mediante dos elementos de entrada: un **potenciómetro**, que regula la velocidad de la secuencia, y un **pulsador**, que invierte su sentido. La placa utilizada es un Arduino UNO y la temporización se maneja con la función **millis()** en lugar de **delay()**, permitiendo que el Arduino siga leyendo las entradas sin bloquear la ejecución del programa.

A diferencia de otros laboratorios, este circuito se desarrolló y probó en **tres fases independientes**, con el objetivo de comprobar por separado el correcto funcionamiento de cada mecanismo de control antes de integrarlos en un único sistema. En la primera fase se probó únicamente el control de dirección mediante el pulsador; en la segunda, únicamente el control de velocidad mediante el potenciómetro; y en la tercera y última fase se combinaron ambos mecanismos en el circuito y el código final.

2. Materiales utilizados

- 1 x Placa Arduino UNO
- 1 x Protoboard (breadboard)
- 1 x Módulo semáforo (LED rojo, amarillo y verde)
- 1 x Módulo pulsador (control de dirección)
- 1 x Potenciómetro (control de velocidad)
- Cables jumper macho-macho y macho-hembra
- 1 x Cable USB (alimentación y carga del programa)
- PC con Arduino IDE instalado

3. Metodología de desarrollo por fases

Para reducir la complejidad de depuración, el sistema se construyó de forma incremental: cada mecanismo de entrada (pulsador y potenciómetro) se implementó y verificó por separado, en su propio circuito y su propio sketch, antes de unirlos en la fase final. Esta estrategia permitió confirmar que la lógica de cambio de dirección funcionaba correctamente de forma aislada, y que el mapeo de velocidad a partir del potenciómetro también funcionaba de forma aislada, de modo que al integrarlos los posibles errores se limitaran únicamente a la combinación de ambos, y no a la lógica individual de cada uno.

4. Fase 1: Prueba con pulsador

En esta primera fase se probó únicamente el control de **dirección** de la secuencia mediante un pulsador, sin potenciómetro. La velocidad del cambio de LED se dejó fija en 300 ms, con el fin de concentrar la prueba en la lógica de inversión de sentido.

Conexiones del circuito (Fase 1)

Componente	Conexión en el Arduino
LED Rojo	Pin digital 13
LED Amarillo	Pin digital 12
LED Verde	Pin digital 11
Pulsador (señal)	Pin digital 2
LEDs y pulsador (referencia)	GND (común)

Código fuente (Fase 1)

El sketch define los pines de los LEDs y del pulsador (pin 2), además de las variables **paso** y **direccion**. En **loop()** se lee constantemente el estado del pulsador, detectando únicamente el flanco de subida (el instante en que se presiona) mediante la comparación con **estadoAnteriorBoton**. Cuando esto ocurre, **direccion** se invierte (multiplicándola por -1) y **paso** se reubica en 0 o en 2 según el nuevo sentido, para que la secuencia continúe de forma coherente. Cada 300 ms (controlado con **millis()**), se apagan todos los LEDs y se enciende el que corresponde al paso actual, avanzando en el orden rojo → amarillo → verde si **direccion == 1**, o verde → amarillo → rojo en caso contrario.

```
int ledRojo = 13;
int ledAmarillo = 12;
int ledVerde = 11;

int pulsador = 2;

int paso = 0;
int direccion = 1;

unsigned long tiempoAnterior = 0;

int estadoAnteriorBoton = LOW;

void setup()
{
  pinMode(ledRojo, OUTPUT);
  pinMode(ledAmarillo, OUTPUT);
  pinMode(ledVerde, OUTPUT);

  pinMode(pulsador, INPUT);
}

void loop()
```

```

{
  // Leer el estado actual del pulsador
  int estadoBoton = digitalRead(pulsador);

  // Detectar solamente cuando se PRESIONA
  if (estadoBoton == HIGH && estadoAnteriorBoton == LOW)
  {
    direccion = direccion * -1;

    // Cambiar el paso para que la secuencia continúe
    // correctamente en la nueva dirección
    if (direccion == -1)
    {
      paso = 2;
    }
    else
    {
      paso = 0;
    }
  }

  // Guardar estado del botón
  estadoAnteriorBoton = estadoBoton;

  // Cambiar LED cada 300 ms
  if (millis() - tiempoAnterior >= 300)
  {
    tiempoAnterior = millis();

    // Apagar todos los LEDs
    digitalWrite(ledRojo, LOW);
    digitalWrite(ledAmarillo, LOW);
    digitalWrite(ledVerde, LOW);

    // Dirección normal: ROJO → AMARILLO → VERDE
    if (direccion == 1)
    {
      if (paso == 0)
        digitalWrite(ledRojo, HIGH);

      if (paso == 1)
        digitalWrite(ledAmarillo, HIGH);

      if (paso == 2)
        digitalWrite(ledVerde, HIGH);

      paso++;

      if (paso > 2)
        paso = 0;
    }
  }
}

```

```

// Dirección inversa: VERDE → AMARILLO → ROJO
else
{
    if (paso == 0)
        digitalWrite(ledVerde, HIGH);

    if (paso == 1)
        digitalWrite(ledAmarillo, HIGH);

    if (paso == 2)
        digitalWrite(ledRojo, HIGH);

    paso++;

    if (paso > 2)
        paso = 0;
}
}
}

```

Evidencia de funcionamiento (Fase 1)

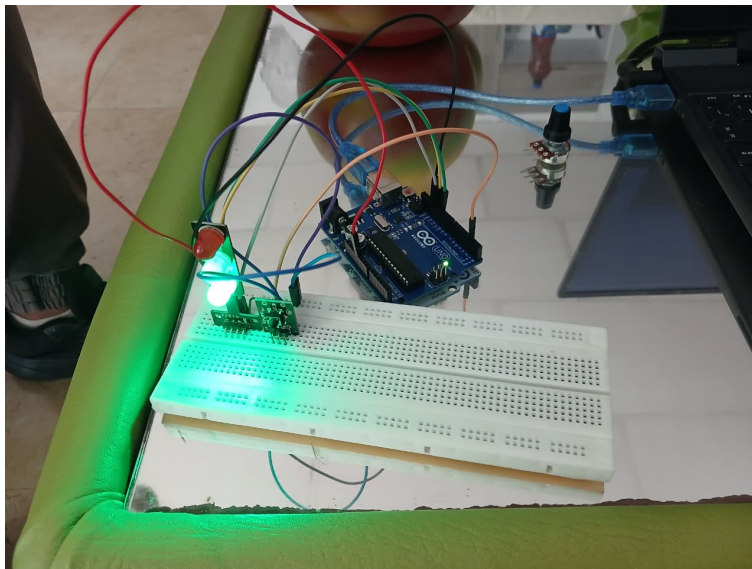


Figura 1. Prueba de la Fase 1: módulo semáforo y módulo pulsador conectados al Arduino UNO sobre la protoboard, con el LED verde encendido durante la secuencia controlada por el pulsador.

5. Fase 2: Prueba con potenciómetro

En esta segunda fase se probó únicamente el control de **velocidad** de la secuencia mediante un potenciómetro, sin pulsador. La secuencia avanza siempre en un mismo sentido (rojo → amarillo → verde), con el fin de concentrar la prueba en el mapeo del valor analógico del potenciómetro hacia un tiempo de espera variable.

Conexiones del circuito (Fase 2)

Componente	Conexión en el Arduino
LED Rojo	Pin digital 13
LED Amarillo	Pin digital 12
LED Verde	Pin digital 11
Potenciómetro (señal analógica)	Pin analógico A0
LEDs y potenciómetro (referencia)	GND (común)
Potenciómetro (alimentación)	5V

Código fuente (Fase 2)

El sketch define los pines de los LEDs y el potenciómetro (entrada analógica A0), además de la variable **paso**. En cada vuelta de **loop()** se lee el potenciómetro con **analogRead()** y su valor (0 a 1023) se convierte mediante **map()** en un tiempo de espera entre 1000 ms (lento) y 100 ms (rápido). Usando **millis()**, cada vez que se cumple ese tiempo se apagan todos los LEDs y se enciende el que corresponde al paso actual, avanzando siempre en el orden rojo → amarillo → verde.

```
int ledRojo = 13;
int ledAmarillo = 12;
int ledVerde = 11;

int potenciometro = A0;

int paso = 0;

unsigned long tiempo = 0;

void setup()
{
  pinMode(ledRojo, OUTPUT);
  pinMode(ledAmarillo, OUTPUT);
  pinMode(ledVerde, OUTPUT);
}

void loop()
{
  int valor = analogRead(potenciometro);

  int velocidad = map(valor, 0, 1023, 1000, 100);

  if (millis() - tiempo >= velocidad)
  {
    tiempo = millis();

    digitalWrite(ledRojo, LOW);
    digitalWrite(ledAmarillo, LOW);
```

```

digitalWrite(ledVerde, LOW);

if (paso == 0)
    digitalWrite(ledRojo, HIGH);

if (paso == 1)
    digitalWrite(ledAmarillo, HIGH);

if (paso == 2)
    digitalWrite(ledVerde, HIGH);

paso++;

if (paso > 2)
    paso = 0;
}
}

```

Evidencia de funcionamiento (Fase 2)

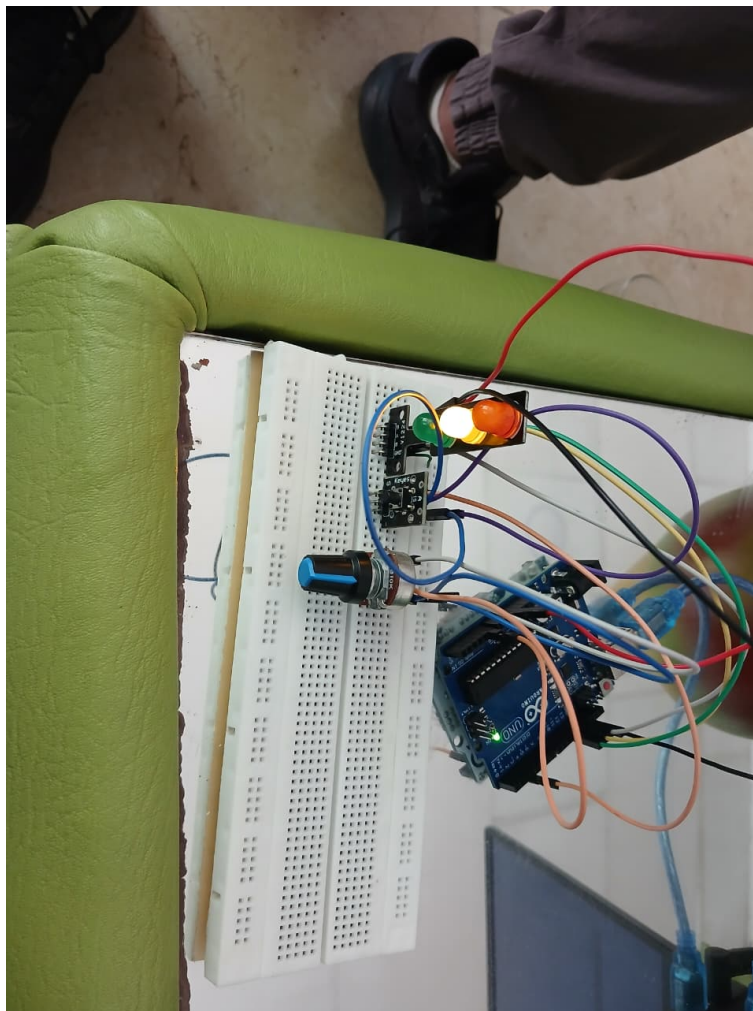


Figura 2. Prueba de la Fase 2: módulo semáforo y potenciómetro conectados al Arduino UNO, con el LED amarillo encendido durante la secuencia cuya velocidad regula el potenciómetro.

6. Fase 3 (Final): Integración pulsador + potenciómetro

Una vez comprobado por separado el funcionamiento del pulsador (Fase 1) y del potenciómetro (Fase 2), en esta tercera fase se integraron ambos mecanismos en un mismo circuito y un mismo sketch: el potenciómetro regula la velocidad de la secuencia y el pulsador invierte su dirección, funcionando de manera simultánea.

Conexiones del circuito (Fase 3 - Final)

Componente	Conexión en el Arduino
LED Rojo	Pin digital 13
LED Amarillo	Pin digital 12
LED Verde	Pin digital 11
Pulsador (señal)	Pin digital 2
Potenciómetro (señal analógica)	Pin analógico A0
LEDs, pulsador y potenciómetro (referencia)	GND (común)
Potenciómetro (alimentación)	5V

Código fuente (Fase 3 - Final)

Este sketch combina la lógica de las dos fases anteriores: por un lado, detecta el flanco de subida del pulsador para invertir **direccion** y reubicar **paso** en 0 o en 2 según corresponda; por otro, lee el potenciómetro y convierte su valor en el tiempo de espera entre cambios mediante **map()**. Usando **millis()**, cuando se cumple ese tiempo se apagan todos los LEDs y se enciende el que corresponde al paso actual, avanzando en el orden rojo → amarillo → verde si **direccion == 1**, o verde → amarillo → rojo en caso contrario, todo mientras el sistema sigue leyendo ambas entradas sin bloquearse.

```
int ledRojo = 13;
int ledAmarillo = 12;
int ledVerde = 11;

int pulsador = 2;
int potenciometro = A0;

int paso = 0;
int direccion = 1;

unsigned long tiempoAnterior = 0;

int estadoAnteriorBoton = LOW;

void setup()
{
  pinMode(ledRojo, OUTPUT);
  pinMode(ledAmarillo, OUTPUT);
```

```

pinMode(ledVerde, OUTPUT);

pinMode(pulsador, INPUT);
}

void loop()
{
    // Leer el estado del pulsador
    int estadoBoton = digitalRead(pulsador);

    // Detectar solamente cuando se PRESIONA
    if (estadoBoton == HIGH && estadoAnteriorBoton == LOW)
    {
        direccion = direccion * -1;

        // Cambiar el paso para continuar correctamente
        // en la nueva dirección
        if (direccion == -1)
        {
            paso = 2;
        }
        else
        {
            paso = 0;
        }
    }

    // Guardar estado del botón
    estadoAnteriorBoton = estadoBoton;

    // Leer el potenciómetro
    int valor = analogRead(potenciometro);

    // Convertir el valor del potenciómetro en velocidad
    // 1000 ms = lento
    // 100 ms = rápido
    int velocidad = map(valor, 0, 1023, 1000, 100);

    // Cambiar LED según la velocidad seleccionada
    if (millis() - tiempoAnterior >= velocidad)
    {
        tiempoAnterior = millis();

        // Apagar todos los LEDs
        digitalWrite(ledRojo, LOW);
        digitalWrite(ledAmarillo, LOW);
        digitalWrite(ledVerde, LOW);
    }
}

```

```

// Dirección normal:
// ROJO → AMARILLO → VERDE
if (direccion == 1)
{
    if (paso == 0)
        digitalWrite(ledRojo, HIGH);

    if (paso == 1)
        digitalWrite(ledAmarillo, HIGH);

    if (paso == 2)
        digitalWrite(ledVerde, HIGH);

    paso++;

    if (paso > 2)
        paso = 0;
}

// Dirección inversa:
// VERDE → AMARILLO → ROJO
else
{
    if (paso == 0)
        digitalWrite(ledVerde, HIGH);

    if (paso == 1)
        digitalWrite(ledAmarillo, HIGH);

    if (paso == 2)
        digitalWrite(ledRojo, HIGH);

    paso++;

    if (paso > 2)
        paso = 0;
}
}
}

```

Evidencia de funcionamiento (Fase 3 - Final)

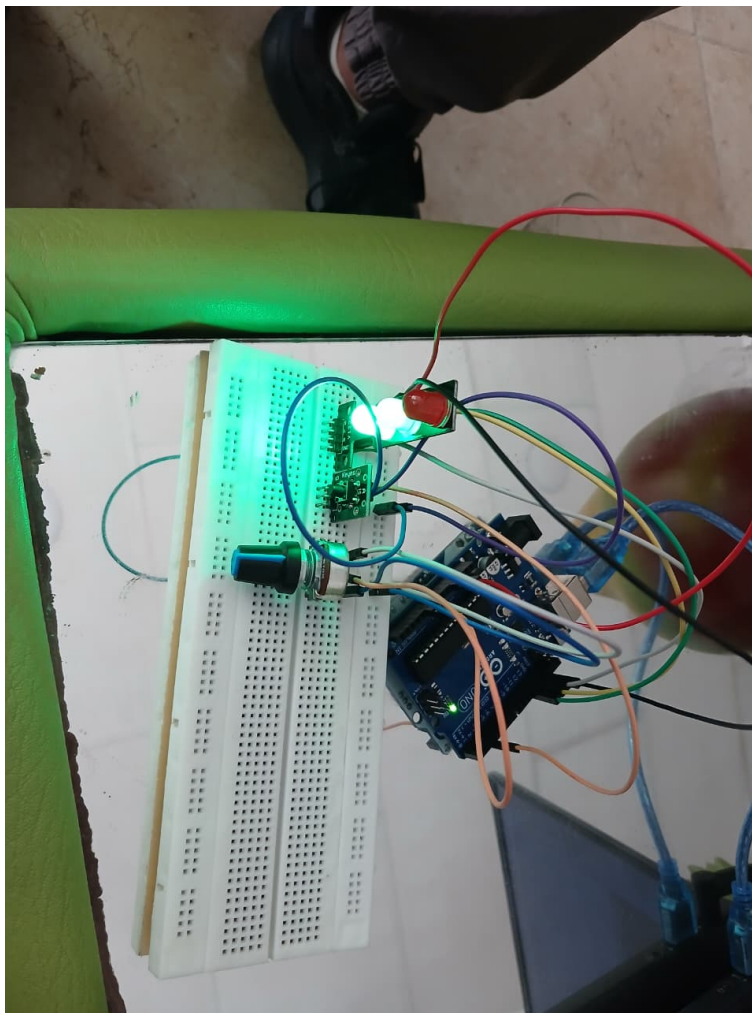


Figura 3. Prueba de la Fase 3 (final): módulo semáforo, pulsador y potenciómetro integrados en un mismo circuito conectado al Arduino UNO, con el LED verde encendido durante la secuencia controlada simultáneamente por ambas entradas.

7. Resultados y conclusión

El desarrollo por fases permitió validar cada mecanismo de control de forma aislada antes de combinarlos: en la Fase 1 se confirmó que el pulsador invertía correctamente el sentido de la secuencia sin repetir ni saltar colores, y en la Fase 2 se confirmó que el potenciómetro ajustaba la velocidad de forma progresiva y sin interrupciones. Al integrar ambos mecanismos en la Fase 3, el sistema final funcionó correctamente: la velocidad y la dirección de la secuencia respondieron de manera simultánea e inmediata a las dos entradas, confirmando que tanto las conexiones físicas como el código fuente de cada fase son correctos.

Posibles mejoras para una siguiente versión:

- Agregar un antirrebote (debounce) por software al pulsador, para evitar lecturas falsas por ruido eléctrico.
- Usar una interrupción externa (`attachInterrupt`) en el pin del pulsador en lugar de sondearlo en cada vuelta de `loop()`.
- Mostrar el valor de velocidad seleccionado por el monitor serial o en una pantalla, a modo de retroalimentación.

- Guardar la última dirección usada en la EEPROM, para que el sistema la recuerde tras reiniciar.
- Declarar los pines con **const int** en lugar de **int**, ya que su valor no cambia durante la ejecución.